

PROJECT AND TEAM INFORMATION

Project Title

Sangam : Your City Your Experience

Student / Team Information

<i>Team Name:</i> <i>Team #</i>	<i>Horizen</i>
Team member 1 (Team Lead)	<i>Dhasmana, Srishti –</i> <i>2510010598</i> <i>dhasmanasrishti715@gmail.com</i> 
Team member 2	<i>Bijalwan, Ishita –</i> <i>2510380152</i> <i>ishitabijalwan.ib@gmail.com</i> 
Team member 3	<i>Rana, Nandini –</i> <i>2510370024</i> <i>nr.nandini21@gmail.com</i> 

PROPOSAL DESCRIPTION

Motivation

Most of us have opened Google Maps or TripAdvisor in a new city and immediately felt a bit overwhelmed hundreds of pins, no obvious order, and no real sense of what actually fits into a single day without doubling back across town. The tools exist to show you what's around you, but very few of them help you turn a list of places into an actual plan. That gap is the starting point for this project: a platform where a user can mark attractions, hotels, and restaurants on a map and get back a usable route, not just a scattered list of markers. A lot of coursework in data structures and algorithms stays abstract — sorting arrays, traversing trees on paper, timing algorithms against each other in isolation. A city map is one of the few everyday things that is genuinely a graph, and route planning is one of the few everyday problems that genuinely needs a shortest-path algorithm and not just a database query. Building this project is a way to apply DSA concepts to something with an obvious real-world shape, rather than to a textbook example.

Existing tools show what's nearby, but they don't help turn a list of places into a usable plan. This project aims to fill that gap: a platform where users can mark attractions, hotels, and restaurants on a map and receive an optimized route instead of scattered markers.

The personal motivation is to apply data structures and algorithms (DSA) to a real-world problem. A city map is naturally a graph, and route planning genuinely requires shortest-path algorithms, making this project a practical application of DSA concepts.

State of the Art / Current solution

The tourism and mapping space is dominated by a handful of large, well-established players, each strong in a different part of the problem:

Google Maps — excellent point-to-point routing and place data, but it optimises for getting from A to B, not for planning a sequence of stops around what a tourist actually wants to see.

TripAdvisor — strong on reviews, ratings, and discovery of attractions, restaurants, and hotels, but weak on route planning; it tells you what's good, not how to see it efficiently.

Google Trips(2019) — was arguably the closest attempt at combining discovery and itinerary building, but Google shut it down and folded parts of it into Google Maps' "lists" feature, which is far less structured.

What's missing across most of these is a visible, efficient way to solve the actual combinatorial problem underneath itinerary planning given a set of places someone wants to visit, what order minimises time spent travelling between them? Commercial platforms either skip this entirely. This project treats that problem as the centre of the application rather than an afterthought, and builds it explicitly using graph algorithms rather than hiding it behind a general-purpose maps API.

Project Goals and Milestones

The main aim is to develop an interactive, accessible platform that allows tourists to explore a city, mark places of interest, and plan their visits efficiently.

To achieve this, the project focuses on the following key points: storing and structuring place data (attractions, hotels, restaurants) in a proper way; enabling quick search of places by name and category; computing optimized routes between selected places, allowing users to browse places by category on an interactive map, and providing a convenient interface to build and manage personalized travel itineraries.

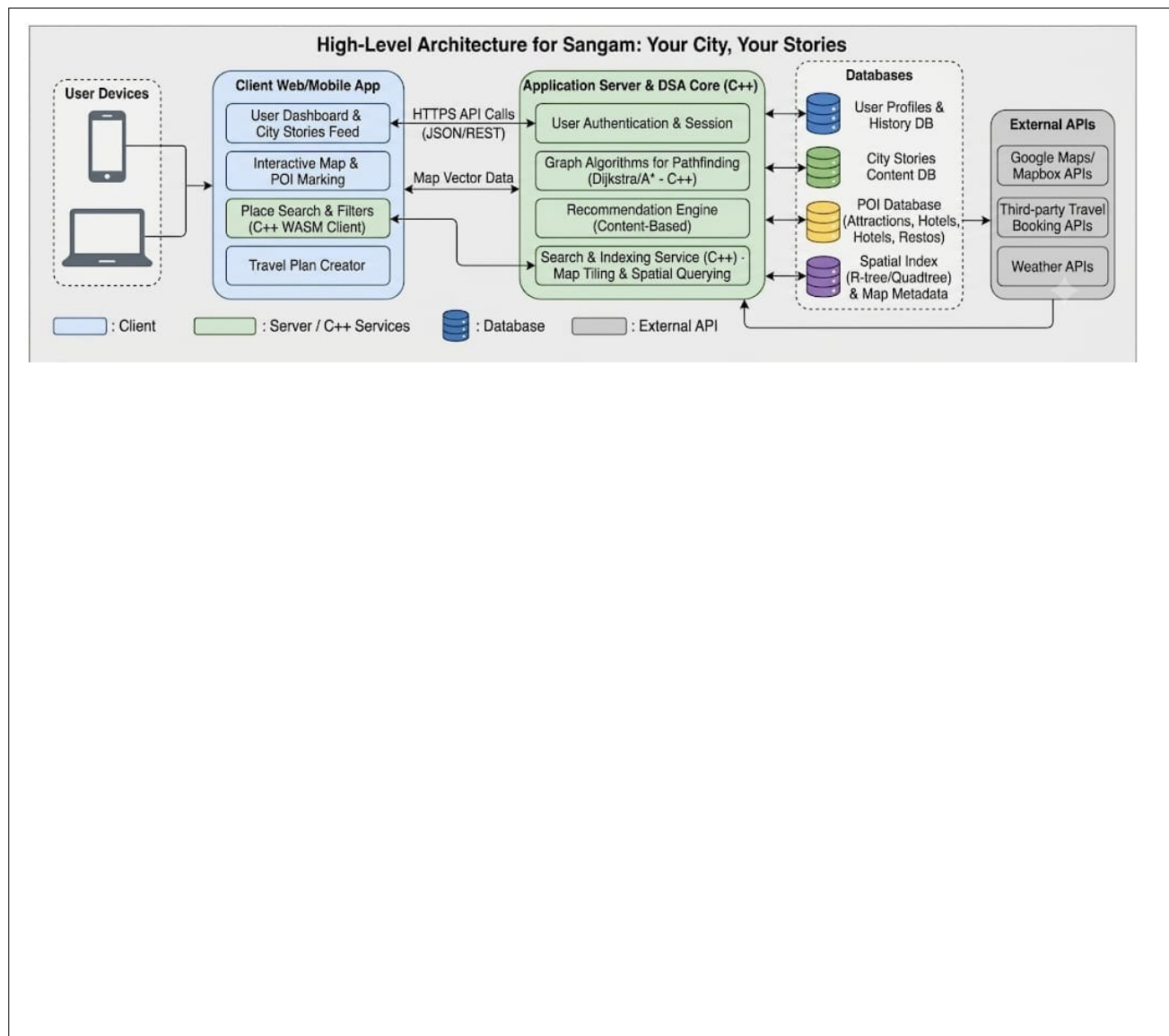
The platform is built around the idea that a city, for planning purposes, is essentially a network of connected locations, and travel planning is essentially a routing problem. Rather than presenting the user with an unordered list of pins, as most existing map and review applications do, the system treats the set of places a user wants to visit as a graph problem and computes a sensible order to visit them in, factoring in distance between stops. Users can add places under three categories, view them on a live map, search for a specific place instantly as they type, and generate a day-wise plan that avoids unnecessary backtracking across the city.

The project involves applying core Data Structures and Algorithms. Arrays and vectors will store information about each place, such as name, description, category, and coordinates. Graphs will represent the city map, with places as nodes and routes as weighted edges. Hash tables will enable fast search by keyword, improving the efficiency of search operations. A trie will support instant prefix-based search as the user types. Priority queues will support shortest-path computation (Dijkstra/A) between places, and a heuristic-based approach will be used for multi-stop itinerary optimization. Linked lists will manage the category groupings, and a lightweight relational database will handle persistence of user data and saved itineraries.*

Planned Milestones

- *Requirement analysis and problem definition*
- *Data collection and structuring of the place database*
- *Implementation of core data structures (graph, hash table, trie)*
- *Implementation of searching and route-optimization algorithms*
- *Implementation of categorized browsing and itinerary-building modules*
- *Integration of the C++ engine with the map interface and database*
- *Testing for correctness, efficiency and usability*
- *Evaluation of the performance of selected data structures and algorithms*
- *Documentation and project presentation*

System Architecture (High Level Diagram)



Project Outcome / Deliverables

One of the main goals of this project is the development of an Interactive City Guide, which will use fundamental data structures and algorithms to organize, search, retrieve, and manage city location data efficiently, while enabling users to explore destinations and generate personalized travel itineraries. The developed program will have the following features:

- *Modeling the city as a graph, where locations such as attractions, hotels, restaurants, shopping areas, and other points of interest are represented as nodes, and the routes or connections between them are represented as weighted edges.*
- *Implementing BFS and DFS traversal techniques to enable exploration of routes and analysis of how different city locations are connected to one another.*
- *Applying shortest-path algorithms such as Dijkstra's or A* to determine the most optimal route between two or more selected destinations.*
- *Storing and managing place-related data using suitable data structures, including arrays, linked lists, trees, or hash tables, selected based on the access and update requirements of the system. Performing efficient searching and sorting operations to help users discover and organize locations according to category, distance, or rating.*
- *Utilizing a priority queue to rank and recommend places based on factors such as user preferences, popularity, or proximity to the user's location.*
- *Designing an itinerary generation module that applies route optimization techniques to construct efficient, personalized travel plans for users.*
- *Ensuring efficient insertion, deletion, searching, and retrieval of location records within the underlying data structures used by the system.*
- *Conducting testing and performance analysis to evaluate time complexity, memory usage, and the overall speed of route computation.*
- *Providing a simple and user-friendly C++ interface for interacting with the system, supported by complete documentation and a final working demonstration.*

Assumptions

1. *The city map used for the project is a simplified, self-contained graph (hand-built/generated from a small OpenStreetMap extract rather than a constantly updating global map)-the goal is to demonstrate correct algorithmic behaviour, not to compete with a production mapping service.*
2. *Real-time factors that commercial map services account for, such as live traffic, road closures, or public transport schedules, are out of scope; all edge weights (distances/times) are treated as static for the duration of a session.*
3. *The user base for testing is assumed to be small (single-machine, a handful of concurrent users at most), so the database layer is built for correctness and clarity rather than for handling heavy concurrent load.*
4. *Place data (names, categories, coordinates, ratings) is either entered manually or drawn from a small pre-collected dataset rather than pulled live from a third-party API, since the focus of the project is the algorithmic and data-structure layer, not data acquisition.*
5. *The web front end is treated as a supporting layer for demonstration purposes; it is not expected to match the polish of a commercial product, since the marks/value of the project sit primarily in the C++ and DSA components.*

References

1. *GeeksforGeeks* – used as a reference for understanding graphs, BFS/DFS traversal methods, Dijkstra’s algorithm, priority queues, and hash table concepts ([geeksforgeeks.org](https://www.geeksforgeeks.org)).
2. cplusplus.com / [W3Schools](https://www.w3schools.com) – referred to for C++ syntax, STL containers, and the fundamentals of object-oriented programming.
3. *OpenStreetMap (OSM)* – consulted for real-world city map data, location coordinates, and the general structure of routes (openstreetmap.org)
4. *Introduction to Algorithms (CLRS)* by Cormen, Leiserson, Rivest, and Stein – used as a theoretical basis for graph algorithms, shortest-path computation, and complexity analysis.
5. *Google Maps Platform Documentation* – referred to for insight into interactive map functionality, place categorization, and how routes and itineraries are typically designed (developers.google.com/maps).